2024 American Control Conference (ACC)
July 8-12, 2024. Toronto, Canada

A Practical Reinforcement Learning (RL) Controller Design for Nonlinear Systems

Hesam Hassanpour[1], Prashant Mhaskar[1], and Brandon Corbett[2]

Abstract— **This paper presents a practically implementable reinforcement learning (RL) approach for process control applications. Standard model-free RL approaches are not applicable in practice because the learning process of an RL agent requires random exploration of state and action spaces, which can compromise process safety and economic objectives. To tackle this issue, an offline training strategy is proposed by leveraging existing model predictive control (MPC) to pre-train the RL agent. MPC actions, calculated offline by solving the MPC optimization problem based on a wide range of initial conditions, along with its objective function are utilized to pre-train the actor and critics of the RL agent. The pre-trained RL controller, with similar performance to the MPC performance, is then utilized for online control for further training. The efficacy of the proposed RL controller to improve the tracking performance is demonstrated using a simulation example for a pH neutralization process.**

I. INTRODUCTION

Advanced model-based control techniques, such as model predictive control (MPC), are predominant methods for control of complex processes in the presence of nonlinearity, uncertainty, and constraints [1], [2]. The performance of such controllers strongly depends on the model accuracy, and therefore changes in model parameters during the operation cycle due to various reasons, such as feedstock variability, catalyst deactivation, fouling, etc., can significantly degrade the closed-loop performance. While re-identification strategies have been developed [3], [4], the high costs and challenges associated with data collection may prevent these techniques from being used in practical situations.

Model-free reinforcement learning (RL)-based control algorithms have recently been developed as alternatives to nonlinear MPC (NMPC) [5], [6]. These controllers have received considerable attention due to their ability to learn the optimal policy directly from online interactions and thus handle changes in operating conditions and unmeasured disturbances [7], [8]. However, random exploration, required for learning process, makes the standard model-free RL techniques inapplicable in process control applications (because of economic and safety risks involved in online exploration). To address this issue, several approaches have been proposed to offline train the RL controller using a surrogate process model [9], [10]. These approaches rely on the availability of accurate nonlinear models, which makes them unimplementable in the absence of such models.

[1]Hesam Hassanpour and Prashant Mhaskar are with the Department of Chemical Engineering, McMaster University, Hamilton, Ontario, Canada. Email: mhaskar@mcmaster.ca

[2]Brandon Corbett is with Sartorius Canada Inc., Oakville, Ontario, Canada.

A limited amount of information (such as step test data) is usually available, in many process industries, to develop a control algorithm. Step-response models have been utilized to implement dynamic matrix control (DMC), which is one the common industrial MPCs [11]. Offset-free feature is also added to handle plant-model mismatch [12], [13]. Although such a linear MPC can provide acceptable performance in some applications, its implementation can be limited when used to control a process over a wide range of operating conditions (the linear model is unable to adequately describe the complex nonlinear dynamics in a wide range of operating regions). The main contribution of this work is to leverage this existing linear offset-free MPC to develop a secure RL controller strategy that is able to handle changes in operating conditions while alleviating regular model maintenance steps, required for successful MPC implementations.

Motivated by the above considerations, this work addresses the problem of designing and implementing a safe RL control strategy by leveraging information from an offset-free MPC (representative industrial MPC). To that end, MPC calculations, obtained offline by solving the MPC optimization problem based on a wide range of initial conditions (to cover state and action spaces), are used to pre-train the actor of the RL controller. The MPC objective function is also leveraged to pre-train the critic networks (offline training). Having obtained a pre-trained RL controller, which can replicate the offset-free MPC performance, it is employed online to interact with the actual process to gain more knowledge about the process nonlinearity, and therefore update and improve the control policy (online training).

The remainder of this paper is organized as follows: The illustrative example, which is a pH neutralization process, is presented in Section II. A brief overview of offset-free MPC and RL is presented in this section. This section also presents an implementation of the offset-free MPC for the illustrative example. The proposed RL control approach, including offline and online training parts, is presented in Section III. Section IV presents the implementation results of the proposed RL control approach for the illustrative example. Finally, concluding remarks are provided in Section V.

II. PRELIMINARIES

A. Illustrative Example: pH Control

The control of pH is an integral part of many processes in chemical and biotechnological industries. It is essential to maintain the pH value within a certain range in order to ensure product quality and safety. However, the pH control

Authorized licensed use limited to: American University of Beirut. Downloaded on February 14,2026 at 09:25:59 UTC from IEEE Xplore. Restrictions apply.

is widely recognized as a difficult problem due to inherent nonlinearities and frequently changing dynamics, which make conventional control methods, such as PID, inadequate to control such processes. RL can be selected because the controller can learn/update the policy based on interactions with the dynamic system. However, these controller-process interactions must be done in a secure manner.

The illustrative example, considered in this work to evaluate the proposed RL controller performance, is a pH neutralization process. The objective is to neutralize a solution of sodium acetate (CH_3COONa) by manipulating hydrochloric acid (HCl) flow rate in a continuous stirred tank reactor (CSTR) [14]. The chemical reaction that occurs in a mixture of sodium acetate with hydrochloric acid is governed by the following equation:

- $CH_3COONa \longrightarrow Na^+ + CH_3COO^-$
- $HCl \longrightarrow Cl^- + H^+$
- $CH_3COO^- + H_2O \Longleftrightarrow CH_3COOH + OH^-$
- $H_2O \longrightarrow OH^- + H^+$

Suppose that hydrochloric acid has a constant concentration, the liquid is a pure dissolution of sodium acetate, the solution is perfectly mixed, and there is no additional buffering effect, the dynamics of the process can be described by the following set of equations [15]:

$$\begin{aligned} \dot{x_A} &= \frac{1}{V}\left(F_A C_A - (F_A + F_S)x_A\right) \\ \dot{x_S} &= \frac{1}{V}\left(F_S C_S - (F_A + F_S)x_S\right) \\ p\dot{H}_m &= \frac{1}{\tau}\left(pH - pH_m\right) \\ -x_A + 10^{-pH} - 10^{pH-14} &+ \frac{x_S}{1+10^{pKS+pH-14}} = 0 \end{aligned} \tag{1}$$

where the ion concentrations are as follows: $x_A = [Cl^-]$, $x_S = [Na^+] = [CH_3COOH] + [CH_3COO^-]$. V is the volume of the CSTR reactor. F_A and F_S are the control acid stream and inlet stream flow rates, respectively. C_A and C_S are the acid concentration in the control acid stream and the concentration of sodium acetate in the inlet stream, respectively. pH_m also represents the measured pH which is subject to time constant τ. The dissociation constants are: $k_w = [H^+][OH^-] = 10^{-14} mol^2/L^2$ and $k_S = [CH_3COOH][OH^-]/[CH_3COO^-] = 5.7143 \times 10^{-9} mol/L$. The process parameters and the steady-state values are listed in Table I.

TABLE I

PROCESS PARAMETERS AND STEADY-STATE VALUES

Parameter	Value	Unit
V	30	L
F_S	1.50	L/min
C_A	0.50	mol/L
C_S	1.20	mol/L
F_{A_s}	2.50	L/min
x_{A_s}	0.31	mol/L
x_{S_s}	0.45	mol/L
τ	1	min
pH_{m_s}	5.40	

As mentioned, the pH neutralization process is highly nonlinear, and an RL-based control strategy can be a good candidate to achieve pH control. However, to develop a practical RL approach, capable of handling random interactions (explorations), a representative offset-free MPC, which will be introduced in the next section, is utilized to pre-train the RL agent.

B. Offset-free MPC

The offset-free MPC formulation is presented in this section [12], [13]. Assume a linear state-space model of the process, given by Eq. (2), is identified:

$$\begin{aligned} x_{t+1} &= Ax_t + Bu_t \\ y_t &= Cx_t + Du_t \end{aligned} \tag{2}$$

where $x \in \mathbb{R}^{n_x}$ is the vector of states, $u \in \mathbb{R}^{n_u}$ is the vector of manipulated inputs, and $y \in \mathbb{R}^{n_y}$ is the vector of outputs. $A \in \mathbb{R}^{n_x \times n_x}$, $B \in \mathbb{R}^{n_x \times n_u}$, $C \in \mathbb{R}^{n_y \times n_x}$, and $D \in \mathbb{R}^{n_y \times n_u}$ are the constant matrices of the model. For offset-free formulation, the model is augmented with a disturbance model to capture the plant-model mismatch in steady state:

$$\begin{bmatrix} x_{t+1} \\ d_{t+1} \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} \begin{bmatrix} x_t \\ d_t \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_t \tag{3}$$

$$y_t = \begin{bmatrix} C & C_d \end{bmatrix} \begin{bmatrix} x_t \\ d_t \end{bmatrix} + Du_t \tag{4}$$

where $d \in \mathbb{R}^{n_d}$ is the vector of augmented disturbances. B_d and C_d are matrices with appropriate dimensions. This augmented model is utilized to implement the offset-free MPC, as follows:

$$\begin{aligned} \min_{u_t,\ldots,u_{t+P}} \quad & \sum_{j=1}^{P} \|y_{t+j} - y_{t+j}^{SP}\|_Q^2 + \|u_{t+j} - u_{t+j-1}\|_R^2 \\ s.t. \quad & x_{t+1} = Ax_t + B_d d_t + Bu_t, \\ & d_{t+1} = d_t, \\ & y_t = Cx_t + C_d d_t + Du_t, \\ & u_{min} \le u \le u_{max} \end{aligned} \tag{5}$$

where y_{t+j}^{SP} is the set-point and y_{t+j} is the predicted output at the time $(t+j)\Delta t$, and Δt is the sampling time. P is the prediction horizon, and Q and R are the weighting (penalty) matrices. $u_{min} \in \mathbb{R}^{n_u}$ and $u_{max} \in \mathbb{R}^{n_u}$ are the lower and upper bounds of the inputs. It should be noted that both the states and disturbances must be estimated in this implementation. Thus, a Luenberger observer is used.

1) Implementation of the offset-free MPC to Illustrative Example: An offset-free MPC is designed to control the illustrative example. We first apply a step test to identify a model of the process. The state-space form of the model, given by Eq. (2), is augmented with a disturbance model. The parameters of the augmented model is listed in Table II. This model is then used to implement the offset-free MPC using Eq. (5) with the following parameters: $\Delta t = 1$

Authorized licensed use limited to: American University of Beirut. Downloaded on February 14,2026 at 09:25:59 UTC from IEEE Xplore. Restrictions apply.

min, $P = 4$, $Q = 3$ and $R = 1$. A Luenberger observer with the gain $L = [9.72,\ 5.03]^T$ is employed to estimate the state and disturbance at each iteration. The manipulated input is also subject to the following constraint: $F_A \in [0, 10]$. Note that the input and outputs are presented in deviation form ($\bar{F}_A = F_A - F_{A_s}$ and $p\bar{H}_m = pH_m - pH_{m_s}$).

TABLE II

MODEL PARAMETERS.

Parameters	Value
A	0.9206
B	0.2399
C	-0.3923
D	0
B_d	0
C_d	1

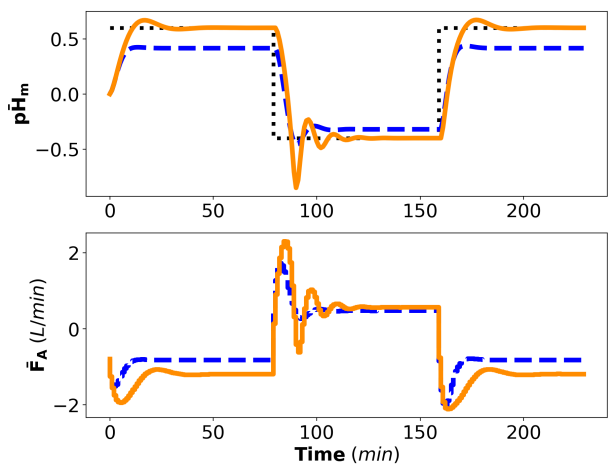


Fig. 1. The output and manipulated inputs obtained based on the MPC (dashed lines) and the offset-free MPC (solid lines), and the set-points (dotted lines).

The offset-free MPC tracking performance is shown in Fig. 1. This performance is also compared with an MPC where the plant model (without integrating the disturbance model) is only used for implementation. While the offset-free MPC shows good capability to track the prescribed set-points, the MPC fails to drive the pH to its set-point because of the existing plant-model mismatch. This offset-free MPC is considered as a representative industrial MPC to design and implement the proposed RL controller.

C. Reinforcement Learning

The model-free reinforcement learning (RL) aims to learn a policy which maximizes the cumulative discounted reward. The RL problem can be modeled as a Markov decision process (MDP), comprised of a state space ($\mathscr{S}$), an action space ($\mathscr{A}$), a reward function ($\mathscr{R}$), and a state transition probability (p). At each time step t, the RL agent observes the current state of the environment, $s_t \in \mathscr{S}$, and then takes an action $a_t \in \mathscr{A}$. By performing this action, the state of the environment changes to a new state, $s_{t+1} \in \mathscr{S}$, and the agent receives a reward $r_t \in \mathscr{R}$ from the environment. The agent uses a policy π to choose the action a_t according to the current state s_t. The final goal is to find the optimal policy that maximizes the discounted accumulated rewards, G_t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \tag{6}$$

where $\gamma \in [0,\ 1]$ is the discount factor.

The expected return, where the agent starts from the state s and follows the policy π is called the state-value function, defined as follows:

$$v_\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | s_t = s \right] \quad \forall s \in \mathscr{S} \tag{7}$$

The action-value function, $q_\pi(s,a)$, which is the expected return, where the agent starts from the state s, takes the action a, and follows the policy π thereafter, is defined as follows:

$$q_\pi(s,a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | s_t = s,\ a_t = a \right] \quad \forall s \in \mathscr{S},\ \forall a \in \mathscr{A} \tag{8}$$

The value functions are utilized to evaluate the policy. The final objective is to find an optimal policy π^* that maximize the value functions. The optimal action-value function q^*, under the optimal policy, can be presented in terms of the optimal state-value function v^*:

$$q^*(s,a) = \mathbb{E}\left[r_{t+1} + \gamma v^*(s_{t+1}) | s_t = s,\ a_t = a \right] \tag{9}$$

These functions satisfy the Bellman equation. Model-based dynamic programming (DP) techniques can be used to find the optimal value function [16]. However, these approaches suffer from the curse of dimensionality, when dealing with large-scale problems. Model-free techniques such as Monte Carlo (MC) and temporal difference (TD) can provide more efficient solutions, in these situations, in the absence of models [16].

Q-learning is a TD-based technique where an iterative algorithm, given by Eq. (10), is used to find the optimal policy by learning the optimal *Q-value* for all state-action pairs.

$$q(s_t,a_t) \leftarrow q(s_t,a_t) + \alpha \left[r_{t+1} + \gamma \max_{a_{t+1}} q(s_{t+1},a_{t+1}) - q(s_t,a_t) \right] \tag{10}$$

where α denotes the learning rate. Learning the *Q-values* for all state-action pairs is challenging for process control applications in the presence of large-scale continuous state and action spaces. Function approximation techniques are developed to address such problems [16]. Deep *Q-network* (DQN) techniques, developed by integrating deep learning with RL, are proposed to address the curse of dimensionality of *Q-learning* in high-dimensional continuous state space [17]. The deep deterministic policy gradient (DDPG) algorithm, which is an actor-critic method, is also developed by

Authorized licensed use limited to: American University of Beirut. Downloaded on February 14,2026 at 09:25:59 UTC from IEEE Xplore. Restrictions apply.

combining deterministic policy gradient (DPG) and DQN to deal with continuous action space problems [18]. In addition, the twin-delayed deep deterministic policy gradient (TD3) method is proposed to tackle the problem of overestimating *Q-values*, observed in DDPG based approaches [19], [20]. In this work, the TD3 algorithm is used to develop the RL controllers.

III. The Proposed RL Controller Design

The proposed RL control approach is presented in this section. It consists of two parts: (1) Offline or pre-training of the RL controller using the offset-free MPC, and (2) Online training of the controller by interacting with the process.

A. Offline training of the RL controller

The offset-free MPC, implemented in Section II-B.1, is leveraged to pre-train the RL controller. The TD3 algorithm, using an actor and two critics, is used. The offset-free MPC optimization problem is utilized offline to calculate the control actions based on a large number of random states and disturbances (x_t and d_t). The generated states and disturbances, together with the first MPC actions u_t^{mpc} are used to calculate the next values of the state and disturbance (x_{t+1} and d_{t+1}), based on the augmented model. These values are also used to predict the next value of the output (y_{t+1}). The reward values are then computed using the reward function, given by Eq. (11). Note that this function is defined based on the offset-free MPC objective function with some modifications. Due to the fact that the RL task is to maximize the reward, the negative of the MPC objective (cost) function (based on the first action) is used.

$$r_t = -(\|y_{t+1} - y_{t+1}^{sp}\|_Q^2 + \|u_t^{mpc} - u_{t-1}\|_R^2) \tag{11}$$

The first MPC actions are used to pre-train the RL controller (the agent's actions are considered to be equal to the offset-free MPC actions during offline training). The current and next states of the RL agent are also defined as follows: $s_t := (u_{t-1}, x_t, d_t, y_t^{sp})$, and $s_{t+1} := (u_t^{mpc}, x_{t+1}, d_{t+1}, y_{t+1}^{sp})$. A collection of the experiences, (s_t, a_t, r_t, s_{t+1}), are then stored in the replay buffer. The objective of the offline training is for the RL actor to compute similar actions to the offset-free MPC. Therefore, we use a mean square error (MSE) loss function in the TD3 algorithm to train the actor (note that this loss function is used just for the pre-training part). The actor is trained to approximate the mapping function $f_{pr}(\cdot)$, as follows:

$$a_t^{pr} = f_{pr}(u_{t-1}, x_t, d_t, y_t^{sp}) \tag{12}$$

The current and next states of the RL agent, the actions of the actor (predicted MPC actions), and the rewards calculated using Eq. (11), are also utilized to train the critic networks using the TD3 algorithm.

B. Online training of the RL Controller

The pre-trained RL controller is deployed online to continue to learn about the underlying process behavior, and therefore, update its policy to achieve improved control. The pre-stored experience buffer, the pre-trained actor, and the pre-trained critics are utilized. The ability of the pre-trained controller to achieve the same performance as the offset-free MPC can provide a safe platform for online implementation. In addition, it can significantly reduce initial random explorations, compared with the standard RL techniques, because the pre-trained controller already knows some possibly sub-optimal state and action regions. This can improve the initial reward values obtained by online implementation of the RL controller. In addition, the pre-trained controller updates its policy as the pre-trained controller interacts with the actual process and gains more knowledge about the process (it gradually moves towards a more optimal policy over time). The experience replay buffer, updated with new experiences from the online interactions, is then used in the standard TD3 algorithm, in order to update the actor and critics. Indeed, the actor is updated to calculate the control actions using the function $f_{on}(\cdot)$, as follows:

$$a_t^{on} = f_{on}(u_{t-1}, x_t, d_t, y_t^{sp}) \tag{13}$$

where a_t^{on} represents the actions during online training. Note that estimation of the states and disturbances are required, based on the augmented model, to calculate the actions, according to Eq. (13). Thus, an observer is employed to estimate these values during online training.

IV. Simulation results

A. Offline training

The already implemented offset-free MPC is utilized to pre-train the RL controller. In this direction, the values of the past inputs, u_{t-1}, current states and disturbances (initial conditions of the augmented model), x_t and d_t, and current set-points, y_t^{sp} are randomized and fed to the offset-free MPC optimization problem (note that these random values are generated in such a way that they cover the entire state and action regions reasonably well). The MPC actions u_t^{mpc} are then computed offline using Eq. (5) with the same parameters as before. The current states of the RL agent are defined using this information. In the next step, the next states of the augmented model, x_{t+1} and d_{t+1}, are calculated by applying the MPC actions, which are used to construct the next sates of the RL agent. Finally, having obtained the values of y_{t+1}, the reward values, r_t, are calculated based on Eq. (11). These experiences are stored in a replay buffer, to be used in the TD3 algorithm to train the actor and critic networks.

A two-layer neural network is used for the actor and critic networks. There are 256 neurons for each hidden layer. The rectified linear unit activation function (*ReLU*) is used for the hidden layers of each network. In addition, the hyperbolic tangent activation function (*Tanh*) is utilized for the output layer of the actor network. The Adam optimizer is used to

Authorized licensed use limited to: American University of Beirut. Downloaded on February 14,2026 at 09:25:59 UTC from IEEE Xplore. Restrictions apply.

update and calculate the parameters of each network. The hyperparameters are also listed in Table III.

TABLE III
HYPERPARAMETERS OF THE RL CONTROLLER

Hyperparameters	Value
Buffer capacity	1×10^6
Actor learning rate	0.001
Critic learning rate	0.001
Batch size	100
Target policy smoothing noise	0.02
Noise clip	0.50
Exploration noise	0.04
Discount factor γ	0.99
Smoothing factor τ	0.005
Policy update frequency	2

Fig. 2 compares the performance of the pre-trained controller with the offset-free MPC. As shown, the pre-trained RL controller can fully replicate the MPC performance. Therefore, this controller can be used as a safe RL controller for online training and implementation.

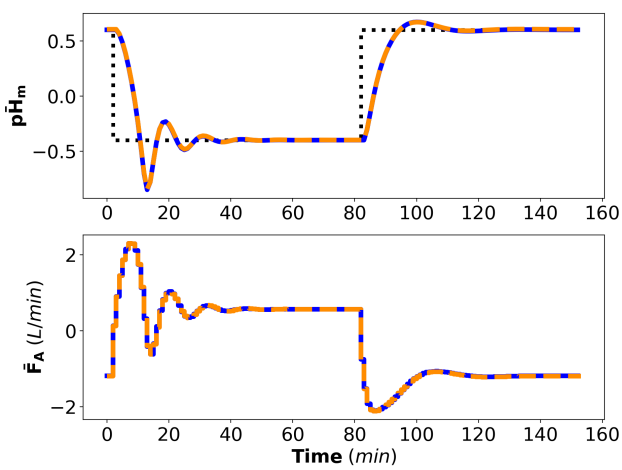
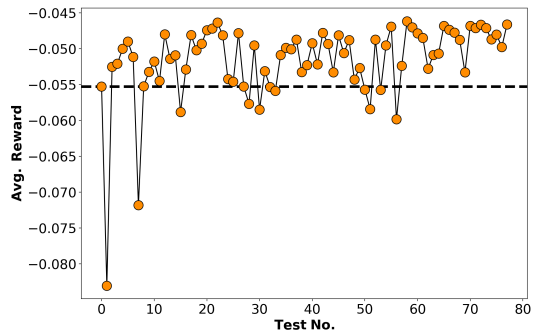


Fig. 2. The output and manipulated inputs obtained based on the offset-free MPC (dashed lines) and the pre-trained RL controller (solid lines), and the set-points (dotted lines).

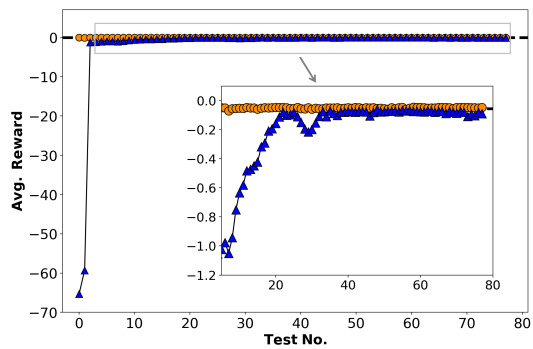
B. Online training

The pre-trained controller is used for online control to interact with the process in order to update and improve its policy by learning more about the underlying nonlinear behavior. The pre-trained actor, critics, and the pre-stored buffer are updated over time as the controller interacts with the process. It should be noted that the same parameters, used for pre-training the RL controller, are also used for online training. Note that training and testing stages are considered to train and evaluate the performance of the online-trained controller. Each training and testing stage consists of 5 and 2 cycles, respectively. Each cycle considers prescribing the first set-point, $p\bar{H}_m^{sp} = 0.6$, for 4 hours, then switching to the second set-point, $p\bar{H}_m^{sp} = -0.4$, for 4 hours. The training and testing process is carried out over a course of six months, resulting in 78 training and testing stages.

The average rewards obtained by implementing the online-trained RL controller, during testing stages, is shown in Fig. 3(a). The horizontal dashed line, in this figure, represents the average reward obtained by implementing the offset-free MPC which is equal to the value achieved based on the pre-trained RL controller. This is considered as a baseline for comparing and evaluating the performance of the proposed approach. As shown, the average reward improves over time compared to the offset-free MPC.



(a)



(b)

Fig. 3. (a) Average reward obtained based on the online-trained RL controller. (b) Comparison of the average reward obtained based on the online-trained RL controller (circles) and the standard RL controller (triangles).

In addition, a standard RL controller is implemented for the illustrative example. The average reward values obtained based on this controller and the online-trained controller are compared in Fig. 3(b). As can be seen, the initial rewards based on the standard approach are very small due to the random explorations to find optimal regions. The agent initially has no information about the process (making the standard RL techniques unimplementable in practice). However, implementing the pre-trained RL controller significantly improve the initial reward values (because the controller is already pre-trained). The average rewards obtained by implementing different approaches during all the testing stages and final testing stage are listed in Table. IV. The results show that the proposed RL controller can improve the average reward values by approximately 7%, for all the testing stages, and 16%, for the final testing stage, compared to the offset-free MPC.

Finally, the tracking performance of the online-trained controller is compared with the offset-free MPC in Fig. 4.

Authorized licensed use limited to: American University of Beirut. Downloaded on February 14,2026 at 09:25:59 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

AVERAGE REWARDS OBTAINED BASED ON THE OFFSET-FREE MPC, STANDARD RL, AND ONLINE-TRAINED RL CONTROLLERS

Controller	Avg. Reward (All Test phases)	Avg. Reward (Final Test)
Offset-free MPC	-0.0552	-0.0552
Standard RL	-1.8246	-0.0906
Online-trained RL	-0.0514	-0.0466

As shown, implementing the online-trained RL controller results in improved closed-loop performance by eliminating the oscillatory behavior, obtained based on the offset-free MPC.

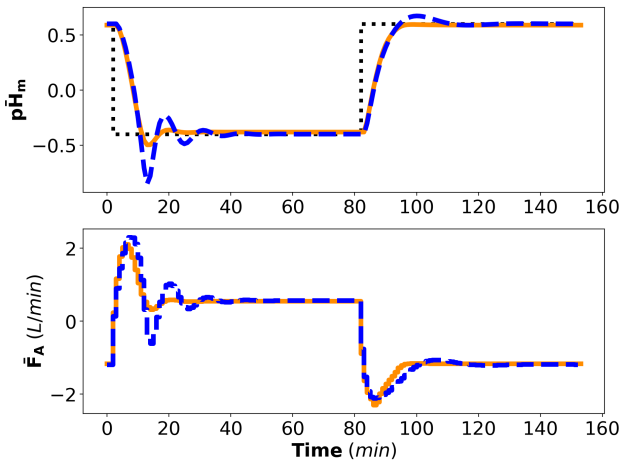


Fig. 4. The output and manipulated input profiles obtained based on the offset-free MPC (dashed lines) and the online-trained RL controller (solid lines), and the set-points (dotted lines).

V. CONCLUSIONS

In this paper, an implementable RL control approach is proposed by leveraging information from a representative MPC (offset-free MPC). MPC computations, obtained offline based on a large number of states and disturbances of the augmented model, are utilized to pre-train the RL actor. The MPC objective function (with some modification) is also used to pre-train the critic networks. Having obtained the pre-trained RL controller, which can replicate the offset-free MPC performance, it is employed online to improve the closed-loop control performance by interacting with the process. The effectiveness of the proposed RL controller is demonstrated using a pH neutralization process. The results show that the proposed RL controller can initially provide a similar performance to the offset-free MPC, and then improve the tracking performance, compared with the offset-free MPC.

ACKNOWLEDGMENT

Financial support from the Natural Sciences and Engineering Research Council of Canada (NSERC) and the McMaster Advanced Control Consortium (MACC) is gratefully acknowledged.

REFERENCES

[1] S. J. Qin and T. A. Badgwell, "A survey of industrial model predictive control technology," *Control Engineering Practice*, vol. 11, no. 7, pp. 733–764, 2003.

[2] D. Q. Mayne, "Model predictive control: Recent developments and future promise," *Automatica*, vol. 50, no. 12, pp. 2967–2986, 2014.

[3] A. H. González, A. Ferramosca, G. A. Bustos, J. L. Marchetti, M. Fiacchini, and D. Odloak, "Model predictive control suitable for closed-loop re-identification," *Systems & Control Letters*, vol. 69, pp. 23–33, 2014.

[4] A. Alanqar, H. Durand, and P. D. Christofides, "Error-triggered on-line model identification for model-based feedback control," *AIChE Journal*, vol. 63, no. 3, pp. 949–966, 2017.

[5] S. Spielberg, R. Gopaluni, and P. Loewen, "Deep reinforcement learning approaches for process control," in *2017 6th international symposium on advanced control of industrial processes (AdCONIP)*. IEEE, 2017, pp. 201–206.

[6] J. Shin, T. A. Badgwell, K. H. Liu, and J. H. Lee, "Reinforcement learning–overview of recent progress and implications for process control," *Computers & Chemical Engineering*, vol. 127, pp. 282–294, 2019.

[7] R. Nian, J. Liu, and B. Huang, "A review on reinforcement learning: Introduction and applications in industrial process control," *Computers & Chemical Engineering*, vol. 139, p. 106886, 2020.

[8] H. Yoo, H. E. Byun, D. Han, and J. H. Lee, "Reinforcement learning for batch process control: Review and perspectives," *Annual Reviews in Control*, vol. 52, pp. 108–119, 2021.

[9] S. Brandi, M. S. Piscitelli, M. Martellacci, and A. Capozzoli, "Deep reinforcement learning to optimise indoor temperature control and heating energy consumption in buildings," *Energy and Buildings*, vol. 224, p. 110225, 2020.

[10] Z. Zou, X. Yu, and S. Ergan, "Towards optimal control of air handling units using deep reinforcement learning and recurrent neural network," *Building and Environment*, vol. 168, p. 106535, 2020.

[11] C. R. Cutler and B. L. Ramaker, "Dynamic matrix control−a computer control algorithm," in *joint automatic control conference*, no. 17, 1980, p. 72.

[12] K. R. Muske and T. A. Badgwell, "Disturbance modeling for offset-free linear model predictive control," *Journal of Process Control*, vol. 12, no. 5, pp. 617–632, 2002.

[13] G. Pannocchia and J. B. Rawlings, "Disturbance models for offset-free model-predictive control," *AIChE Journal*, vol. 49, no. 2, pp. 426–437, 2003.

[14] M. J. Fuente, C. Robles, O. Casado, S. Syafiie, and F. Tadeo, "Fuzzy control of a neutralization process," *Engineering Applications of Artificial Intelligence*, vol. 19, no. 8, pp. 905–914, 2006.

[15] F. Tadeo, O. P. Lopez, and T. Alvarez, "Control of neutralization processes by robust loop shaping," *IEEE Transactions on Control Systems Technology*, vol. 8, no. 2, pp. 236–246, 2000.

[16] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.

[17] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, and S. Petersen, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[18] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[19] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," *In Proceedings of the AAAI conference on artificial intelligence*, vol. 30, no. 1, 2016.

[20] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International conference on machine learning*. PMLR, 2018, pp. 1587–1596.

Authorized licensed use limited to: American University of Beirut. Downloaded on February 14,2026 at 09:25:59 UTC from IEEE Xplore. Restrictions apply.